Structures in C

Matteo Fasiolo

Today

1. Why group related variables?
2. Define, initialise and use structure variables.
3. Use structures as function arguments.
4. Combine structures with fixed-size and dynamically allocated arrays.

Grouping variables

We have learned how to declare **individual** variables:

- `double pi = 3.141592654;`
- `int student_id = 195235;`

In many applications, several variables describe one thing and should be kept together.

For example:

- a student ID, name, nationality, year of study and mark;

A program should handle these related values together.

Example: vector operations

Print a vector:

```
void print_array(double values[], int length) {

    for (int i = 0; i < length; i++) {

        printf("%.2f ", values[i]);

    }

    printf("\n");
}
```

We are passing the array and its length separately.

Declare and initialise a vector:

```
double a[] = {1.0, 2.0, 3.0};

int length = 3;
```

Pass the array together with its length:

```
print_array(a, length);
```

An array and its length are **related** and must agree.

Passing related values separately is inconvenient and error-prone.

Grouping variables

It would be better to keep the array and its length together:

```
print_vector(v1);
```

where `v1` contain elements and a length.

How can we group related values in C?

C provides a language feature for this: a **structure**.

A structure keeps several related variables together.

Structure types and members

A **structure type** groups related variables, called **members**.

Define a structure type:

```
struct structure_name {
    data_type member1;
    data_type member2;
    // ...
}; // Do not forget the semicolon
```

Declare a variable of that type:

```
struct structure_name variable_name;
```

Access one of its members with `.`:

```
variable_name.member1
```

Example: student

We want a new structure of type "student".

Define the structure type with members `id`, `year` and `mark`:

```
struct student {
    int id;
    int year;
    double mark;
};
```

Declare a variable, then assign each member:

```
struct student matteo;
matteo.id = 1024;
matteo.year = 1;
matteo.mark = 70.0;

printf("%.1f\n", matteo.mark);
```

Alternatively, initialise all members when declaring the variable:

```
struct student matteo = {1024, 1, 70.0};
```

The list in braces can be used during initialisation, but not later:

```
struct student matteo;

matteo = {1024, 1, 70.0}; // invalid
```

If you write:

```
struct student matteo = {1024, 1};
```

this initializes the first two members, and sets `mark` to zero.

Alternatively, you can also write:

```
struct student matteo = {.id = 1024, .mark = 70.0};
```

which does not rely on the order. Here `year` is set to zero.

Passing structures to functions

You can defined functions that take structure parameters.

```
void change_mark(struct student s) {
    s.mark = 99.0;
}
```

Structure arguments are copied.

So the parameter `s` is a local variable and

```
struct student matteo = {1024, 1, 70.0};

change_mark(matteo);

printf("%.1f\n", matteo.mark); // prints 70.0
```

does not have any effect outside of `change_mark`.

Let's look at `structures_demo.c` in VS Code.

Structures with arrays members

Back to our vector example.

Store a three-element array directly inside the structure:

```
struct vector {
    int length;
    double elements[3];
};
```

Then, declare a variable of type `struct vector` and use it:

```
struct vector v;
v.length = 3;

for (int i = 0; i < v.length; i++) {
    v.elements[i] = i + 1.0;
}
```

The array is part of `v`, and its size is fixed in the definition.

Now the printing function has only one parameter:

```
void print_vector(struct vector v) {
    for (int i = 0; i < v.length; i++) {
        printf("%.2f ", v.elements[i]);
    }
    printf("\n");
}

// Later in main()
print_vector(v);
```

Note: `print_vector` **copies the whole structure**, including the array.

So a function such as

```
void modify_struct(struct vector v) {
    v.elements[0] = 99.0;
}
```

does not change the original vector when called

```
struct vector x = {3, {1.0, 2.0, 3.0}};
modify_struct(x);
```

If you want to change the members of a structure either:

1. you pass only the array member
2. pass a pointer to the whole structure

1. You pass only the array member

```
struct vector {
    int length;
    double elements[3];
};

// OR void modify_elements(double elements[])
void modify_elements(double *elements){
    elements[0] = 99.0;
}
```

and later:

```
struct vector v = {3, {1.0, 2.0, 3.0}};

modify_elements(v.elements);
```

Now `v.elements[0]` is changed to `99.0`.

2. Pass a pointer to the whole structure

```
struct vector {
    int length;
    double elements[3];
};

void modify_elements(struct vector *v){
    (*v).elements[0] = 99.0;
}
```

and later:

```
struct vector v = {3, {1.0, 2.0, 3.0}};

modify_elements(&v);
```

Now `v.elements[0]` is changed to `99.0`.

A clarification on pointers and structures:

```
void modify_elements(struct vector *v){
    (*v).elements[0] = 99.0;
}
```

We dereference the pointer `v` to get the structure, then access its member `elements` with `.`

A shortcut is `v->elements[0] = 99.0;`

Similarly, `v->length` is equivalent to `(*v).length`.

This `*v.elements` is wrong:

- the `.` operator has higher precedence than `*`, so
- `v.elements` evaluated first, but `v` is a pointer, it does not have members.

To avoid mistakes use `v->elements` if `v` is struct pointer.

Structures with dynamic array members

What if the number of elements is known only while the program runs?

```
struct vector2 {
    int length;
    double *elements;
};
```

The structure stores a pointer; the array can be allocated separately:

```
struct vector2 v2;
v2.length = count;
v2.elements = malloc(v2.length * sizeof(double));

for (int i = 0; i < v2.length; i++) {
    v2.elements[i] = i + 1.0;
}
```

Use and release the dynamic vector

```
void print_vector2(struct vector2 v) {

    for (int i = 0; i < v.length; i++) {

        printf("%.2f ", v.elements[i]);
    }

    printf("\n");
}
```

```
print_vector2(v2);

free(v2.elements);
```

Copying v2 copies its length and pointer, not the separately allocated array.

```
struct vector2 {
  int length;
  double *elements;
};

void modify_struct(struct vector2 v) {
  v.elements[0] = 99.0;
}
```

and later in `main()`:

```
struct vector2 v2;
v2.length = count;
v2.elements = malloc(v2.length * sizeof(double));

modify_struct(v2);
```

Now `v2.elements[0]` is changed: pointer is copied, not the array.

This is different from the fixed-size version (see above)!

Fixed-size and dynamic versions

Fixed `vector`	Dynamic `vector2`
array stored inside structure	structure stores a pointer
size fixed in the definition	size selected at runtime
no `malloc` or `free`	use `malloc` and `free`

Now let us compare the two versions in a live demonstration.

Let's look at `vector.c` and `vector2.c` in VS Code.

Conclusion

1. A structure type groups related variable together.
2. Use `.` to access a member of a structure variable.
3. A structure argument is copied into the function's local parameter.
4. An array can be stored inside a structure or allocated separately and reached through a pointer.